Chemometrics and Intelligent Laboratory Systems, 20 (1993) 129–148
Elsevier Science Publishers B.V., Amsterdam

Orthonet: orthogonal latent variable neural network

Barry J. Wythoff [1]

Inorganic Analytical Research Division, Chemical Science and Technology Laboratory, National Institute of Standards and Technology, Gaithersburg, MD 20899 (USA)

(Received 9 November 1992; accepted 9 March 1993)

Abstract

Wythoff, B.J., 1993. Orthonet: orthogonal latent variable neural network. *Chemometrics and Intelligent Laboratory Systems*, 20: 129–148.

A methodology has been developed for performing stepwise orthogonal decomposition of a data set using a derivative of the backpropagation neural network algorithm. The network is divided into two parts: a linear orthogonal feature extraction portion, and a nonlinear mapping portion consisting of one or more layers. Error feedback from the nonlinear mapping portion is used to direct the feature extraction process. In addition, the use of multivariate Gaussian kernel function nodes has been generalized to allow inclusion as any backpropagation network node. The derivation of these algorithms is described, and some of their properties are illustrated with controlled artificial data distributions.

INTRODUCTION

The trend of scientific instrumentation in the twentieth century has been one of a steady increase in the number of measurements obtained on the subjects under study. This increase has come about through technological advances which make possible ever higher resolution, bandwidth, and spectral range, as well as the recent trend toward combining several techniques in one experiment — the use of so-called 'hyphenated methods'. As a result, the task of data reduction facing modern scientists is a fundamental one. Data reduction is often important because it is difficult to extract meaningful information from large amounts of raw data. Additionally, data reduction can reduce the 'cost' of subsequent computation, storage, and transmission of information. The efficiency of these processes can be the determining factor in the technological or economic feasibility of an application. Finally, data reduction can greatly facilitate empirical mathematical modeling of data where an insufficient number of samples exists to properly constrain the parameters in a proposed model.

Motivation

A practical task motivated the development of the methods reported in this manuscript. That

[1] Present address: Molecular Structure Characterization Unit, Analytical Technology Division, Kodak Research Laboratories, Eastman Kodak Co., Rochester, NY 14560-2132, USA.

task was the determination of what functional groups are in a given unknown molecule, using infrared spectral data. It was decided that the ability to flexibly use Gaussian nodes in a neural network would likely enhance the performance of the resulting system. For this purpose, a generalized backpropagation training rule for multivariate Gaussian (MVG) nodes was derived. Once this was done, a new problem had been introduced: the dimensionality of the spectral input vectors to the Gaussians was so large as to require an excessive number of weights in the MVG weight matrices. The desire to reduce the dimensionality of the input in a way that was 'appropriate' for a given network model and training set resulted in the Orthonet algorithm. The application of Orthonet to this functional group classification will be described in an upcoming publication.

MAPPING PROBLEMS AND ORTHOGONAL DATA REDUCTION

We are concerned here with the problem of mapping a set of measured 'input data' (e.g., infrared spectra) onto a set of correlated 'output data' or properties (e.g., functional group class). Although the mapping is to be formulated from a set of known inputs and corresponding outputs (a training set), the true goal is to be able to use the 'learned' mapping to estimate the desired output properties for unknown inputs. It can be assumed that some amount of random as well as deterministic 'noise' has colored the training inputs and outputs, and that the training set contains only a fraction of the input–output pairs of interest.

The training inputs may be represented by an $m \times n$ matrix $\mathbf{X}$, where m is the number of measured patterns, and n is the number of measured properties. The training outputs may be similarly represented by an $m \times l$ matrix $\mathbf{Y}$, where l is the number of properties of interest. A model equation: $\mathbf{Y} = F(\mathbf{X}) + \epsilon$ with adjustable parameters in F will be 'fitted' to the training samples to obtain the mathematical transfer rule.

For various reasons, which may include noise reduction, numerical stability, interpretability, and calculation time or storage requirements, we wish to carry out a preliminary stage in this procedure. This stage will consist of choosing k orthogonal features, to represent $\mathbf{X}$, where k is $\leqslant n$. The feature set can also be expressed as an $n \times k$ matrix $\mathbf{P}$. Projecting the input data onto $\mathbf{P}$ will therefore produce a transformed input set, $\mathbf{X}' = \mathbf{XP}$, and the output will be estimated with $F(\mathbf{X}')$. (The reader is cautioned that $\mathbf{X}'$ refers here to the transformed input, not the transpose of the input.) The overall problem is therefore separated conceptually into two parts: (1) choosing an appropriate basis set $\mathbf{P}$, and (2) parameterizing the function, $F(\mathbf{X}')$, so as to obtain a smooth generalization of the relationship between the sample patterns in $\mathbf{X}'$ and $\mathbf{Y}$.

Fourier analysis [1], principal components analysis (PCA) [2] and partial least squares analysis (PLS) [3] are the primary tools currently used to carry out an orthogonal reduction of an input matrix (e.g., a set of spectra). Although there are many differences in these techniques, we choose to focus here on the extent to which they use information from the application problem. The Fourier transform has a fixed basis set consisting of sine and cosine functions at integral multiples of a base frequency. The only flexibility in this system is the option to select portions of the full Fourier basis set, as is done in 'low pass filtering'. If the properties of interest are directly related to the periodic components defined by the Fourier basis set, then Fourier analysis is an appropriate choice. Otherwise, it will not be a very efficient means to reduce the data volume, or to promote higher level understanding of the data structure.

Principal components basis vectors, or 'loading vectors', are defined by the input data, and the input data alone. The directions of the basis vectors are defined so as to preserve the maximum amount of the variance present in the input. While this is a reasonable criterion, it is not necessarily germane to all the purposes for which PCA is actually applied. If the properties of interest correspond directly to the major variations in the input, then PCA will provide useful information. Otherwise, it may not.

Partial least squares analysis uses a compromise criterion which utilizes information from

both the input and output matrices to define the basis set. PLS identifies directions which balance two objectives of maximizing the variance retained in $\mathbf{X}'$, and minimizing the errors in predicting $\mathbf{Y}$ from $\mathbf{X}'$ [4]. In its most basic form, strictly linear (planar or flat) relationships between $\mathbf{X}'$ and $\mathbf{Y}$ are optimized. The vast majority of chemical applications have used this form of PLS. Recently, some extensions of PLS to polynomial and nonlinear relationships have been reported in the chemical literature [5–10].

ORTHONET AND NONLINEAR PLS

Orthonet shares some of the philosophy and characteristics of the nonlinear PLS algorithm. From a neural network perspective, one of the defining characteristics of Orthonet is that it uses a first hidden layer employing only linear transfer functions in the nodes, and that the weight vectors in these nodes (analogous to loading vectors in PCA and PLS) are orthogonal to one another. These first layer nodes will hereafter be termed 'feature nodes', and their weight vectors, 'feature vectors'. The portion of the network 'downstream' from the feature extraction layer will be called the 'functional mapping layers'. There have been other neural net algorithms employing an orthogonal linear first hidden layer reported in the literature [11–13].

Hrycej [11] devised a separate two step process for training the network. In the first stage, a 'backward inhibition' algorithm is used to simultaneously learn the principal components of the input via a 'connectionist' method. In the second stage, the feature nodes are frozen, and the nonlinear mapping portion is trained to reproduce the outputs, using the 'scores' (here $\mathbf{X}'$) as a static set of input features.

In a subsequent generalization of the above method, Hrycej [12] devised modifications such that the feature vectors correspond to an arbitrary rotation of the principal components of the input. It is important to note that rotation of the principal components will not change the portion of the input information which is carried on them. There are two main distinctions to be made between the backward inhibition methods and Orthonet. The first is that Orthonet uses a stepwise, rather than a parallel algorithm to arrive at the features. The second is that the Orthonet features are not restricted to the principal components [11], or to a linear combination of them [12]. As is often the case, whether Orthonet or backward inhibition will be more useful for a given problem will depend on the data set, and on the measure of 'usefulness'.

Qin and McAvoy [13] developed a PLS–neural network hybrid which uses a neural network to calculate the so-called 'inner relation' between the $\mathbf{X}$ and $\mathbf{Y}$ scores. In their scheme, the feature vectors (loading vectors) were derived without using any feedback from the nonlinear inner relation. Holcomb and Morari [14] have reported another PLS–neural network hybrid which does use the nonlinear modeling feedback to help determine the optimal feature weights. Both of these neural network–PLS hybrids use a stepwise approach involving one-dimensional nonlinear models between the input and output projections. Likewise, both utilize an objective which contains a driving force for large input projections as a part.

In a mathematical sense, the goals of this work were: (a) for a given nonlinear model describing the relationship between an arbitrary $\mathbf{X}$ and $\mathbf{Y}$, to be able to select the 'optimal' k-dimensional subset of the input space to carry out the prescribed mapping; (b) to use the minimum number of features, k, needed to explain $\mathbf{Y}$; (c) to determine the 'optimal' parameters for the nonlinear function $F(\mathbf{X}')$. 'Optimal' was chosen here as those parameters which minimize the mean squared error (MSE) between the actual network outputs and the desired network outputs:

$$E = \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{q} \left(y_{ij} - \hat{y}_{ij} \right)^2 \tag{1}$$

In considering the differences between the nonlinear PLS and Orthonet, it should be noted first that no requirement was included in the objective function that maximum input variance be preserved. The reason for this is that some mapping problems, e.g., infrared spectra to func-

tional group class, can depend on features contributing only a small portion of the overall variance in the input. Determining the presence or absence of a nitrile functional group, for example, must involve emphasis of the region 2215–2275 cm^{-1} [15]. In a large and general infrared spectral library, only a small fraction of the spectra will contain absorption bands in this small spectral region. Those bands which do appear there will likely be of only moderate intensity. This region of the spectrum will therefore account for a only a very small fraction of the total input variance.

While the preceding paragraph is just one example, it is not so uncommon to be interested in an occurrence which is either relatively rare and/or small in magnitude in comparison to the total measured data variance. In such a case, focussing more tightly on output prediction should produce more compact latent variable models.

It should be noted that it is possible to include in Orthonet the tendencies of PLS or PCR for maximizing the input variance. Such components are typically more robust features in the presence of noise [4]. One could simply augment the standard mean squared error objective function with a subtractive term related to the ability of the feature vectors to describe the input. A logical choice would be the squared projections of the inputs on the current feature node:

$$E = a_1\left(\sum_{i=1}^{m}\sum_{j=1}^{q}\left(y_{ij}-\hat{y}_{ij}\right)^2\right) - a_2\left[\sum_{i=1}^{m}\left(\sum_{j=1}^{n}x_{ij}p_j\right)^2\right] \quad (2)$$

The feature nodes would have to be kept normalized to unit length to avoid their length playing a role in determining Eqn. 2. By adjusting the relative values of the scalar constants a_1 and a_2, one would obtain a continuous variation in the solution reflecting different priorities on the ability of the features to explain either **X** or **Y**. This would provide a flavor of 'continuum regression' [16] to neural network solutions, including Orthonet. Since the two terms in the objective function (2) are independent, it is relatively straightforward to derive the necessary feature weight derivatives required for Orthonet. For the work reported in this initial manuscript, however, the simpler objective function (1) is used. The use of the augmented objective function (2) along with further variations aimed specifically at classification will be illustrated in a subsequent manuscript.

A second major difference between Orthonet and the nonlinear PLS algorithms appearing in the chemical literature is that more complex models can be formed with Orthonet. Nonlinear PLS algorithms to date have either used a sum of one-dimensional models, or have used nonlinearities in the input only ($F(\mathbf{X}')$ linear in the adjustable parameters). Orthonet employs multidimensional models which can be nonlinear in the parameters. It is speculated here that for a given model complexity, the same mapping error can be achieved on many data sets by using more compact multidimensional models than is achievable by summing one-dimensional models. In other words, it is proposed here that a smaller number of features can often be used if we use a multidimensional model incorporating all the features simultaneously, rather than a series of one-dimensional models incorporating each feature in turn.

The third distinction between any of the neural network models and nonlinear PLS may seem pedantic to some readers: the neural network paradigm can sometimes serve to allow a 'high level' (in the computer science sense) description of the problem to be used. The network, after training, then becomes an executable symbolic representation of the solution. The reason that this may seem pedantic to some is that in principle, complex models can be implemented with either means. In practice, very complex neural network solutions have been achieved, where it is doubtful that the scientists would have proposed an equivalent model using a purely analytical approach.

Finally, a practical distinction can be made between the group of neural network/orthogonal basis set algorithms and the nonlinear PLS algorithms. The neural network methods utilize a computer implementation of the nonlinear portion that is relatively simple, since modular pro-

gram libraries implementing standard node function building blocks are easily established. The programmer only needs to solve the differential equations relating to the simple building block transform equations.

BACKPROPAGATION NETWORKS AND THE FOUNDATION FOR ORTHONET

There are numerous tutorial articles and general information texts on the subject of backpropagation or 'backprop' networks as well as other connectionist models [17–20]. We will concern ourselves here only with those characteristics relevant to the present modifications to the existing body of work.

Feedforward, backpropagation networks are layered structures of interconnected processing nodes (Fig. 1). Each node may have a different functional form for producing the single output value from its inputs. Backpropagation network nodes typically have the following form: first, a weight vector with a corresponding element for each of the node inputs is used to calculate the net input for the ith node:

$$a_i = \sum_{j=1}^{n} x_j w_{ij} = \boldsymbol{x} \cdot \boldsymbol{w}_i \tag{3}$$

Then the node output is obtained by carrying out some differentiable transformation on the node input:

$$o_i = f(a_i), \quad \text{often } o = \frac{1}{1+\mathrm{e}^{-a}} \tag{4}$$

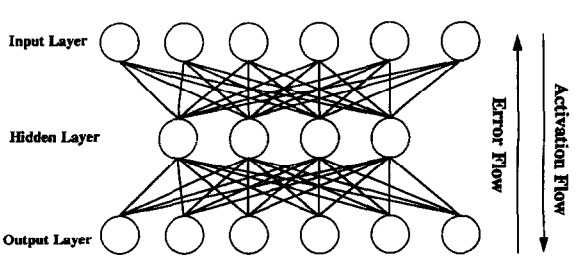

Fig. 1. A schematic diagram of a feedforward, backpropagation network employing a single hidden layer (adapted from ref. 7).

A common form for the sigmoidal transfer function is shown in Eqn. 4. This transfer function is used so often in layered feedforward nets that for many scientists the term backpropagation network implies the use of sigmoidal nodes. Although such nodes are useful as an 'all-purpose' network component, it should be obvious that this one function cannot provide the most compact (or robust) solution for all training problems. Any differentiable node functions may be used. Node functions should be selected using any a priori knowledge about the solution 'shape', with the goal being to select matching node function shapes. Gemperline et al. [21], for example, employed linear and quadratic transfer functions in addition to sigmoids for chemical mixture quantitation.

A backprop network will have at minimum an input layer and an output layer. The input layer performs no transformation of the input patterns (**X**), but serves only to distribute them to the next layer's nodes. The output layer node outputs provide the network estimation of the desired output properties (**Y**). Sandwiched between these two layers may be one or more 'hidden layers'. We are concerned here with a particular type of architecture: one which contains a first hidden layer composed of k linear nodes (feature nodes). For these nodes, the transfer function is simply the identity function:

$$o_i = a_i \tag{5}$$

The weight vector for the ith feature node corresponds to the ith column of the **P** matrix described earlier. If k is less than the rank of the input data matrix, then the feature extraction layer must select a subset of the full space spanned by the input data. This fact imposes a constraint which is important to the Orthonet algorithm.

The network weights (including the feature vectors) are regression coefficients which are adjusted using gradient descent so as to minimize the chosen cost function. Gradient descent requires that moves be made according to the nega-

tive of the local gradient of the error surface:

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}, \quad \Delta w_{ij} = -k\frac{\partial E}{\partial w_{ij}} \tag{6}$$

The derivation of the learning rules is carried out in the appendix; it is appropriate, however, to present a few items here. Following the derivation of Rumelhart and McClelland [19], the weight error derivative is expanded:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_i}\frac{\partial o_i}{\partial a_i}\frac{\partial a_i}{\partial w_{ij}} \tag{7}$$

In the case where the net input is derived from Eqn. 3, the last term of Eqn. 7 is given by:

$$\frac{\partial a_i}{\partial w_{ij}} = x_{ij} \tag{8}$$

The consequences of Eqn. 8 are critical to the Orthonet algorithm. This equation represents the coupling of the error gradient sign and magnitude to the spatial domain of the input. It shows that the relative proportions of the gradient magnitude are the same as those of the input, hence the directional character of the gradient is the same as that of the input. Considering the weight changes for a node to comprise a vector, the learning relationship can be stated as:

$$\boldsymbol{w}_i = \boldsymbol{w}_i + \Delta \boldsymbol{w}_i, \quad \Delta \boldsymbol{w}_i = \sum_{j=1}^{m} c_j \boldsymbol{x}_j \tag{9}$$

This equation states that the weight change vector for the ith node will be given by a linear combination of the input vectors presented during that training cycle. The coefficient for the jth input pattern (c_j) is given by the error term(s) fed back to the ith node from the output node(s). Therefore, a weight vector that is initialized into the space spanned by the inputs will be confined to that space during learning. Orthonet sequentially learns the directions of the $\boldsymbol{P}$ feature vectors. After each feature vector is learned, the input data are orthogonalized with respect to it. Subsequent feature vectors which are drawn initially from the space *currently* spanned by the inputs are thereby guaranteed to be orthogonal.

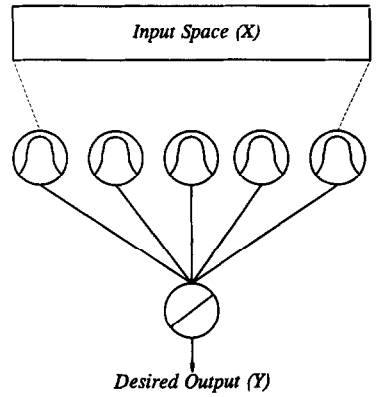

Fig. 2. A schematic diagram of a simple radial basis function network.

MULTIVARIATE GAUSSIAN NODES AND BACKPROPAGATION

We will digress now for a moment to consider the use of multivariate Gaussian nodes in backpropagation networks. Such nodes produce a localized response region in their input space. The use of a linear combination of Gaussian functions to empirically approximate smooth functions belongs to the class of methods called kernel approximation methods (so-called 'potential methods' are a subset of these). Coomans et al. [22] reported the use of such a scheme for chemical pattern recognition in the early 1980s. So-called 'radial basis function' (RBF) networks (Fig. 2) [23] can be considered to be a generalization of earlier kernel function approximation methods. The simplest scheme employs radially symmetric kernel functions with a homogenous width which are centered on each training pattern. The use of a smaller number of kernels, with centers distributed according to the probability distribution of the input patterns, usually provides better generalization properties. Further refinements include the use of heterogenous width parameters for the different nodes, and different widths along each input dimension for each node. The centers and widths may be adjusted by least squares or other methods.

For classification problems, the response shape of Gaussian nodes is particularly desirable when the objects corresponding to data classes are con-

fined to localized (rather than open-ended) regions in the input space. Note that the clusters themselves need not have a Gaussian distribution to make the use of Gaussian nodes profitable. An ensemble of Gaussian kernel nodes can be combined in unlimited ways with neural networks to effect a kernel function approximation to the desired function. It is true that closed, approximately Gaussian-like response regions can be implemented in an n-dimensional space by summing $2n$ sigmoids per region [17]. This requires careful 'coordination' of the sigmoids during training, which becomes increasingly difficult with the number of dimensions. The complexity of the error surface increases in general with the number of nonlinear nodes used. Therefore, if closed response regions are desired, it is sensible to begin with building blocks possessing this character.

In addition to the Orthonet feature extraction system, a generalized backpropagation training algorithm for multivariate Gaussian nodes was derived for this work. This was done by extending the efforts of Saha et al. [24] toward 'oriented nonradial basis function networks'. Saha et al. presented a derivation for a two-layered network in which a single layer of multivariate Gaussians is sandwiched between the input layer and a linear output layer. Their work represented a generalization of RBF networks to use multivariate Gaussian kernels. The new derivation presented in the Appendix is placed in the framework first adopted by Rumelhart and McClelland [19], and permits multivariate Gaussian nodes to be used in a backpropagation network, in any layer, and mixed in arbitrary ways with other node transfer functions on the same or different layers.

Each multivariate Gaussian node requires an $n \times (n+1)$ weight matrix, for an n-dimensional input vector. This second-order scaling dependency creates the potential for enormous nonlinear optimization problems for even moderately large applications. A problem with 1000 input variables for which we wanted to apply 100 multivariate Gaussian nodes would require over one hundred million weights. Even a single optimization run would stretch the limits of current computer technology. In addition, it is highly improbable that sufficient data samples would exist to properly constrain so many adjustable parameters. Rather than simply selecting or averaging variables or even using PCA to achieve variable and parameter reduction, Orthonet can be used to construct the optimal abstract feature set for the data and network structure at hand.

'ONE-DIMENSIONAL' ORTHONET

Consider now the network shown in Fig. 3, which employs a single linear node in the first hidden layer, and a single Gaussian node in the output layer. For this net, the feature layer can present only a one-dimensional subset of the input to the Gaussian functional mapping layer. The network task is to dichotomize the input, i.e., to classify inputs into two groups A and ¬A (we will make use here of the more compact logical symbol for the negation operator: read ¬ as NOT). The correct outputs for input patterns are assigned 1.0 and 0.0, respectively. In order to minimize the error, several things must happen. The description of the input passed by the feature extraction node ($\boldsymbol{x}'$) must be different for A and ¬A. Furthermore, the Gaussian node in the output layer places constraints on the shape of the minimum error distribution of the $\boldsymbol{x}'$ for the patterns belonging to class A: for one thing, they should fall into a single cluster. Since the linear node and Gaussian weight matrix together perform an affine transformation on the input data,

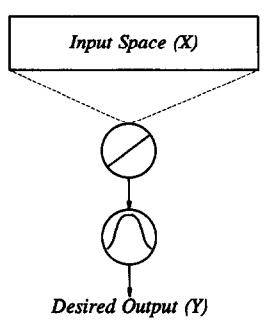

Fig. 3. A backpropagation network passing the input through a linear 'bottleneck' node.

a subset of the full Gaussian response may be selected to both 'match' and influence the distribution of the data on the feature extraction node. (An affine transformation is just a generalization of a linear transformation which includes a 'translation' or 'offset' term.) As the feature extraction node axis is rotating in the input data space, the Gaussian node will track the position and size of the cluster corresponding to class A in its own one-dimensional input space. The Gaussian node will direct the rotation of the axis through the error backpropagation process. At the conclusion of a successful error minimization, the feature extraction node will have found a direction which minimizes the MSE when a one-dimensional Gaussian classification function is employed in this subspace.

MULTI-DIMENSIONAL ORTHONET

If the network performance is unsatisfactory after training with one feature node, it may be proposed to simply add more linear nodes to the first hidden layer. Unfortunately, there is little in the backpropagation algorithm to enforce linear independence among simultaneously trained nodes in this layer. Therefore, a high degree of redundancy could result from a conventionally trained linear feature layer. Linear independence is a minimal requirement for nonredundancy of the feature nodes. Orthonet imposes the stricter orthogonality condition on them.

Recall that Eqn. 8 specifies that changes to the weights will occur only within the space spanned by the inputs for a node. Therefore, if a feature vector is initialized to a direction within the space spanned by the inputs, then it will be 'confined' to that space during learning. Orthonet develops the feature node weight vectors in stepwise fashion, exploiting this property. At the conclusion of each feature node training process, the projections of the input along that feature node are removed by an orthogonalization step. Although the rank of the input data is reduced by one during this step, the number of input variables remains the same. The input data are therefore constrained to an ever decreasing subspace of the original n-dimensional input space.

PREPROCESSING

In order to ensure that the weight vectors lie within the space of the input patterns, it is required that either (a) the input be reduced to full rank, or (b) feature vectors be initialized using a combination of one or more pattern vectors. This will also improve learning efficiency. A simple way to reduce a matrix to full rank is to reduce it to row echelon form [25]. The rows containing nonzero elements can then serve as a minimal basis set. In some cases, it may be appropriate to use PCA to perform a preprocessing data reduction. A highly conservative approach must then be taken when discarding components with small eigenvalues. Removal of any independent deterministic information from the data would reduce or eliminate benefits from a subsequent Orthonet feature extraction.

ORTHONET ALGORITHM

We will now consider the detailed 'recipe' for the Orthonet algorithm. Only one feature extraction node is trained at a time. Previously determined features are simply 'memorized'. All extracted information is, however, presented to the functional mapping layers. The reader should refer to the schematic diagram in Fig. 4 to help understand the algorithm.

(1) Allocate a new feature extraction node, and augment the weight structures of the functional mapping layers as necessary to accommodate a new input.
(2) Initialize the new feature extraction node weight vector to a direction within the span of the current input data matrix.
(3) Train the network using backpropagation until satisfactory convergence is obtained.
(4) If the current mapping error is acceptable, go to EXIT.
(5) Memorize the current feature vector, as well as the outputs for the input data vectors on

the current feature node. These latter values will be supplied to the functional mapping layers by a new 'phantom node'.

(6) From the input data vectors, subtract the component along the feature extraction node. This reduces the rank of the input matrix by one.

(7) Allocate a new phantom node, and relocate the weights between the functional mapping layer and the feature extraction node to the new phantom node.

(8) Go to (1).

(9) EXIT.

At the conclusion, the feature vector set is defined by the last feature extraction node trained, plus all the phantom nodes. Step (6)

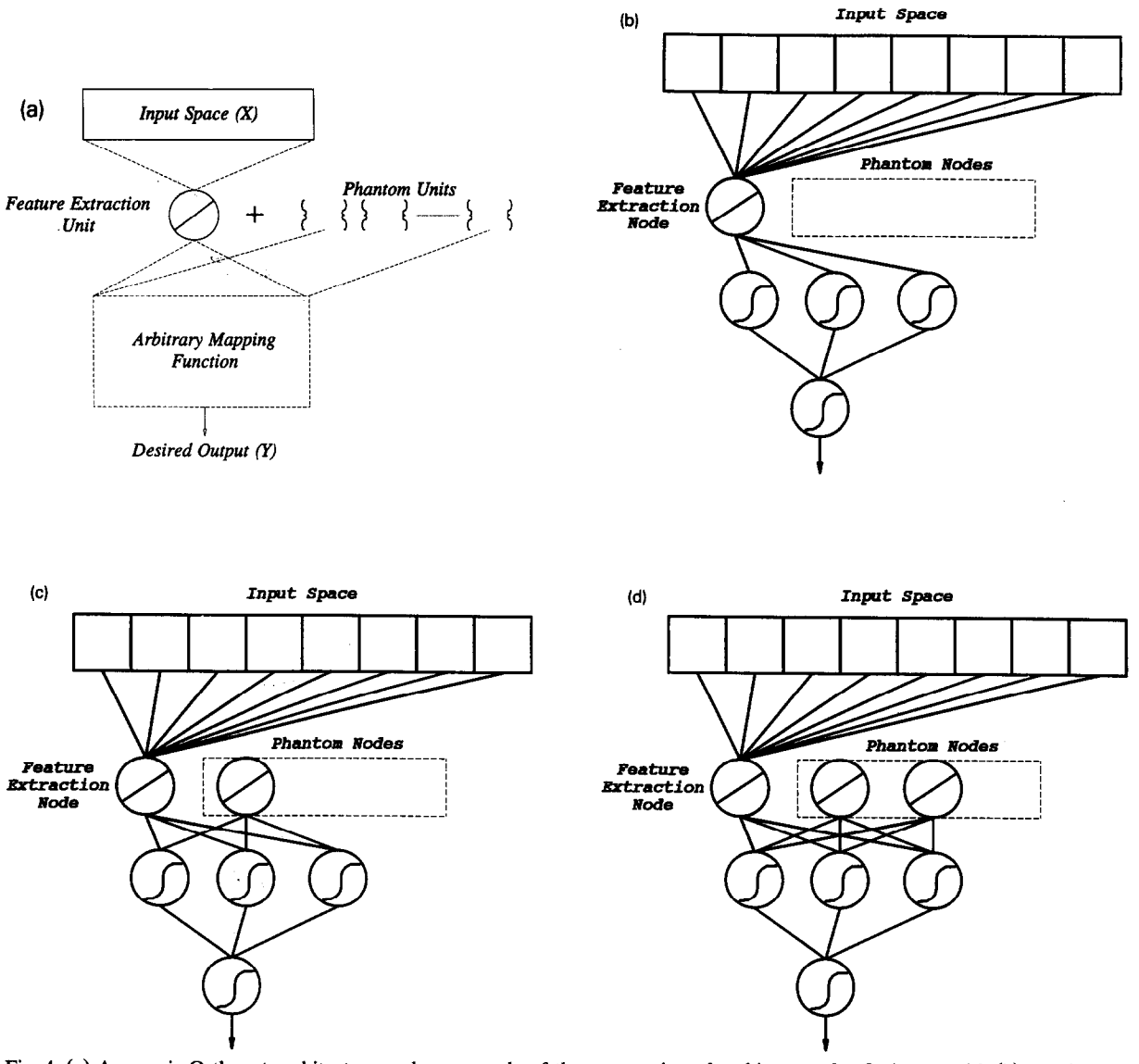

Fig. 4. (a) A generic Orthonet architecture and an example of the progression of architecture for Orthonet, with (b) one feature extracted and no phantom nodes. (c) Two features extracted, and one phantom node. (d) Three features extracted and two phantom nodes.

above can be stated for the ith feature node and jth input pattern as:

$$x_j \leftarrow x_j - \left[\left(\frac{x_j \cdot w_i}{\| w_i \|^2} \right) w_i \right] \tag{10}$$

DISCUSSION ON THE ORTHONET ALGORITHM

The phantom nodes may seem mysterious at first. Each one is derived from a trained feature node, and its output is produced by simple recall of the outputs which were measured for the training set on the corresponding feature node when it was finished training. In terms of the introductory discussion, the output of phantom node j for ith input pattern is just x'_{ij}, analogous to a PLS 'score' for a trained loading vector. There are no input weights to the phantom nodes to be adapted. A feature node, however, does have its output weights transferred along with it, when it becomes a phantom node. When feature node $k+1$ is added, the parameters involved in $(F(x'_j), j = 1, k)$ are thereby used as the starting point for the optimization of $(F(x'_j), j = 1, k+1)$.

The stepwise nature of the algorithm provides diagnostic information in the progression of mapping error with the number of extracted features, and ensures that a parsimonious representation of the input is found. Furthermore, the stepwise aspect provides an explicit ordering of the features in terms of their ability to capture the information essential to predicting the desired output. Finally, the stepwise nature helps to tie the individual feature vectors to specific factors in the data. If they were developed simultaneously, an infinite number of equivalent feature vector sets could be found which would span the same subset of the input space.

Although the feature vectors are defined one at a time, there is still a simultaneous aspect to the Orthonet algorithm. The nonlinear functional mapping layers always operate in the full subspace of the extracted features, rather than just in the current one-dimensional feature extraction node space. There are several reasons for doing this. First, inclusion of all extracted information allows the functional mapping layers to converge to their final form simultaneously with the feature extraction process. Secondly, we speculate here that certain nonlinear mappings require conjoint (i.e., multidimensional) presentation of the input to the nonlinear function, rather than successive one-dimensional presentations, in order to be solved. It is recognized that the fact that the individual features are still fixed, once learned, can be a liability for some problems. The hybrid stepwise and simultaneous nature of Orthonet is an attempt to address some of the concerns relating to joint presentation of the selected input space, while retaining the numerous advantages of a stepwise system discussed earlier.

The stepwise algorithm presented here for Orthonet could be called Orthonet-II, since it is actually the second incarnation which was developed. The first method, Orthonet-I, simultaneously develops all the orthogonal feature vectors through a method that is much simpler to implement and understand. Orthonet-I uses a static number of feature nodes in the first hidden layer which are trained simultaneously. They are kept orthogonal to one another by following each weight change step with a Gram–Schmidt orthogonalization [25] on the first hidden layer. Although this scheme is crude, it does allow simultaneous development of orthogonal feature vectors.

There are at least two serious problems in the associated learning dynamics, however. First, consider all the weights in the feature extraction layer to constitute one weight vector. In Orthonet-I, we move this vector first according to a direction correctly indicated by the error gradient. The Gram–Schmidt step then imposes a second move which is arbitrary with regard to the error surface shape. Secondly, there is an inherent hierarchy in the Gram–Schmidt procedure. Vectors which are processed first will be changed the least, and those which are processed last will be changed the most. Both of these problems will worsen as the number of feature vectors is increased. The concept behind this design was that training might still be successful if these weight vectors are moved only in very small steps.

Orthonet-I was implemented and applied to a

complex problem motivating the developments reported in this manuscript — classification of organic functional groups using infrared spectra. While some successful results were obtained, the convergence rate was poor, and deteriorated rapidly when more than a couple of feature vectors were used. It is worth noting that Holcomb and Morari used simultaneously developed feature vectors in their PLS–neural approach [14] which they also kept orthogonal using the Gram–Schmidt algorithm. They decoupled the training of the feature layer and the nonlinear mapping portion of the network, and reported good convergence rates, presumably because they changed the feature layer much more slowly than the nonlinear mapping portion.

There is yet another variation to Orthonet which, while not yet implemented, could be implemented with minor modifications, and may prove interesting as well. Orthonet-III would modify the output after each stage as well as the input, and would not use phantom nodes. The output would be modified after training each feature extraction node so as to subtract the portion modeled with that node. Such a network would therefore act at each stage to predict the output residuals from the previous stage. The final system could be used for prediction by simply summing the outputs of the trained stages. This Orthonet-III algorithm would be even closer to a classical PLS type of design, and would be significantly simpler to implement than Orthonet-II.

Orthonet-I fits the nonlinear functional map in the full, simultaneously developed n-dimensional subset of the input. Orthonet-III would fit successive one-dimensional nonlinear functions in one-dimensional subsets of the input. Orthonet-II is a hybrid which fits a single nonlinear function in a stepwise expanding subset of the input.

ILLUSTRATIVE EXAMPLE PROBLEMS

In order to demonstrate some of the properties of this new technique, it was decided to consciously design a low-dimensionality artificial data set with controlled properties. Similarly, the network architectures employed here are very simple — the reason for this is to allow conscious joint design of the data set and architectures so as to reveal the desired properties. It is important that the reader understands that the point of the following examples is not to demonstrate that they *can* be solved by Orthonet. The motivation for these examples is to illustrate *how* they are solved by Orthonet.

Four classification tasks were designed which made use of Gaussian data clusters in an eight-dimensional space. Each data cluster was composed of 250 objects. The object coordinates were obtained by using a random number generator producing a Gaussian distribution of outputs with mean 0.0 and standard deviation 1.0. This was generalized with an af fine transformation of the standard Gaussian deviates, to allow an arbitrary mean and standard deviation to be obtained. In all cases, the task was a binary classification task, where the network was asked to produce an output value of 1.0 if the input pattern belonged to the target class, and otherwise, to produce a 0.0.

A simple modification to the basic mean squared error function was employed. To give equal importance to minimization of errors on members of the target class A, which appeared less frequently in the data set than ¬A, weighted errors were used:

$$wt_{\mathrm{A}} = \frac{Num_{\neg \mathrm{A}}}{Num_{\mathrm{t}}} \tag{11}$$

and

$$wt_{\neg \mathrm{A}} = \frac{Num_{\mathrm{A}}}{Num_{\mathrm{t}}} \tag{12}$$

where wt_{A} is the weight applied to errors from patterns containing class A; $wt_{\neg \mathrm{A}}$ is the weight applied to errors from patterns not containing class A; Num_{A} is the number of patterns containing class A in the training set; $Num_{\neg \mathrm{A}}$ is the number of patterns not containing class A in the training set; Num_{t} is the total number of patterns in the training set.

Batch mode training of backpropagation networks requires that the estimates of the error

TABLE 1

Parameters characterizing the Gaussian distributions for the eight-dimensional artificial data sets

Cluster number *	Mean/Std. devn.		Other variables
	Variable 4	Variable 8	
1	0.00/0.50	0.00/0.25	0.00/0.50
2	1.50/0.50	1.50/0.25	0.00/0.50
3	1.50/0.50	−1.50/0.25	0.00/0.50
4	−3.00/0.25	0.00/0.50	0.00/0.50

* Cluster 1 indicates the cluster corresponding to the target group. Clusters 1–3 made up data set IA; clusters 1–4 made up data set IB.

gradient be obtained by summing the individual contributions of the errors measured on a subset of the training set, before making a move on the error surface. Such a batch constitutes a training 'epoch'. As the error derivatives are accumulated, the sum of the individual pattern weights encountered in the epoch is also tallied. This weight sum is then used to normalize the accumulated error derivatives, instead of the batchlength, as is normally done. Other than the normalization, the fractional complement error weighting method is equivalent to that employed by Curry and Rumelhart [26].

Data set IA contained one target data cluster, and two 'interfering' data clusters. Data set IIA was produced by adding a third interfering data cluster to IA. Table 1 shows the mean and standard deviation parameters for these clusters in IA and IIA. These parameters are the same for six of the eight dimensions. Variables four and eight, however, have differences assigned. The projection of data set IA on variables four and eight is shown in Fig. 5a, to illustrate the meaningful structure for classification purposes. An example of the remaining structure is shown in Fig. 5b, which depicts the projection of data set IA on variables one and two.

Two similar network architectures were employed in this study. They employed eight input nodes, a linear feature extraction layer, a second hidden layer consisting of either a single sigmoid or a single Gaussian, then an output layer consisting of a single sigmoid (Fig. 6). The net using a sigmoid in the second hidden layer will be termed the 'sigmoid–sigmoid' net, and the other the 'Gaussian–sigmoid' net. The sigmoid in the output layer was chosen for the benefit of the Gaussian–sigmoid net, and conveys a great deal of added flexibility to the shape of the overall response of the Gaussian–sigmoid network.

The equations used for the net input and the output for the sigmoidal nodes were as shown in Eqns. 3 and 4, respectively. The equation used for the multivariate Gaussian nodes was

$$o_i = e^{a_i} \tag{13}$$

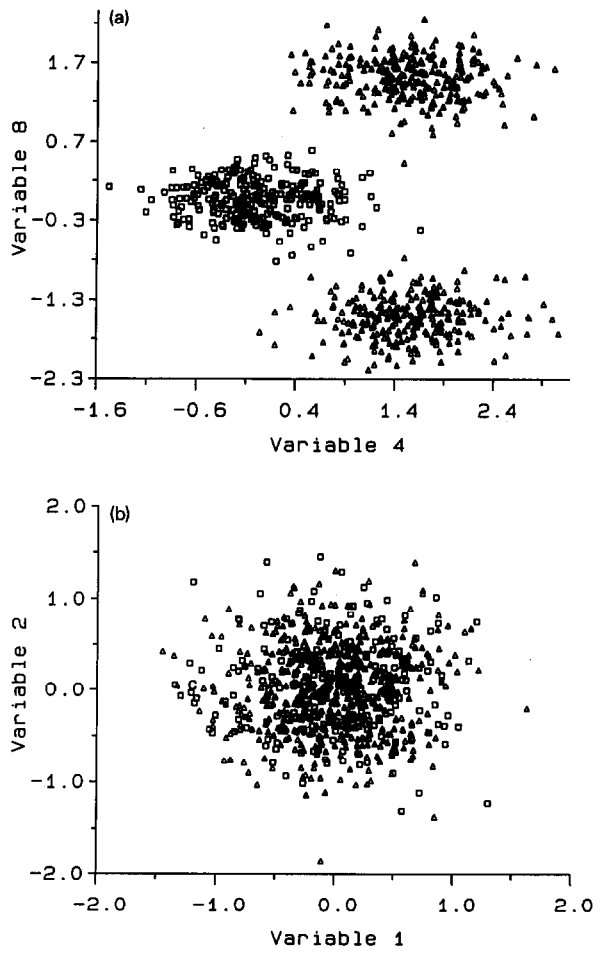

Fig. 5. Data set IA, shown projected onto (a) variables 4 and 8, the two 'meaningful' variables for classification and (b) variables 1 and 2, examples of the remaining overlapped variables. The target cluster is shown with squares and interfering clusters with triangles.

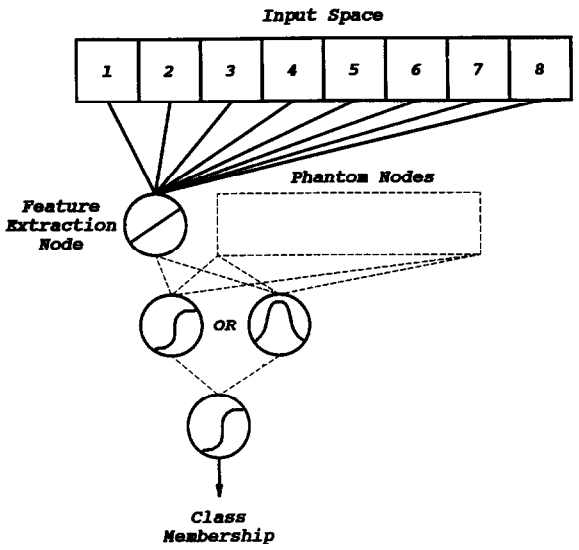

Fig. 6. A schematic diagram of the architectures used to solve the example problems in the text. The second hidden layer was composed of either a single sigmoid or a single Gaussian node, constituting the sigmoid–sigmoid and Gaussian–sigmoid architectures, respectively.

and

$$a_i = -(\mathbf{M}_i \boldsymbol{x})^{\mathrm{T}}(\mathbf{M}_i \boldsymbol{x}) \tag{14}$$

where $\mathbf{M}_i$ is the weight matrix of the node i; $\boldsymbol{x}$ is the input vector to the node i. Note that in each of the above equations the input vector is augmented with a constant extra element equal to 1.0. Use of this constant as the last element in the input vector, along with the corresponding element in the weight vector of the sigmoid and elements in the weight matrix of the Gaussian, generalizes the response to have an arbitrary center. In a sigmoidal node weight vector this element is often referred to as an 'offset', and referenced separately in the equation.

Note also that the equation for the Gaussian response is not the same as that for the multivariate normal distribution, since it lacks the factor which normalizes the integral of the latter. The multivariate normal distribution is an explicit and inclusive model for a probability distribution. Therefore, the integral of the probability must be 1.0 when summed over all space. Multivariate Gaussian nodes are used in a neural network to build up complex empirical solutions to smooth function approximation problems. This more general application does not require normalization.

Some of the flexibility gained by coupling the Gaussian to a downstream sigmoid is conveyed in Fig. 7. For the purposes of discussing these plots, we will express the Gaussian response in the equivalent form for the univariate case:

$$o = \exp\left\{-\left[(x - x_0)/\sigma_x\right]^2\right\} \tag{15}$$

Fig. 7a depicts the effect of changing the σ parameter of the Gaussian function from 2.0 to 5.0, in steps of 1.0. The sigmoidal weight and offset were fixed at 10.0 and −8.0, respectively, for this plot. Fig. 7b depicts the effect of changing the

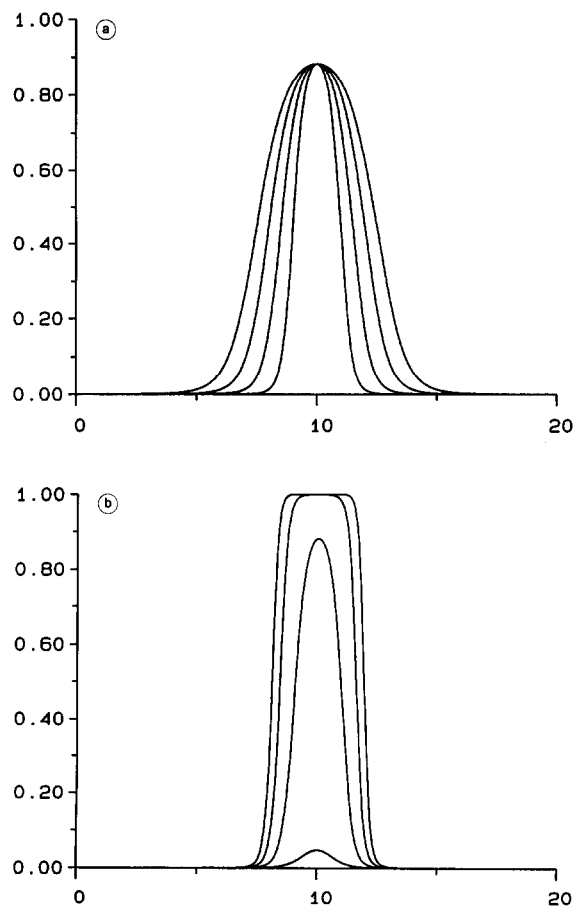

Fig. 7. The overall response of a univariate Gaussian–sigmoid system, when (a) the Gaussian width parameter is varied and (b) the sigmoidal amplification parameter is varied.

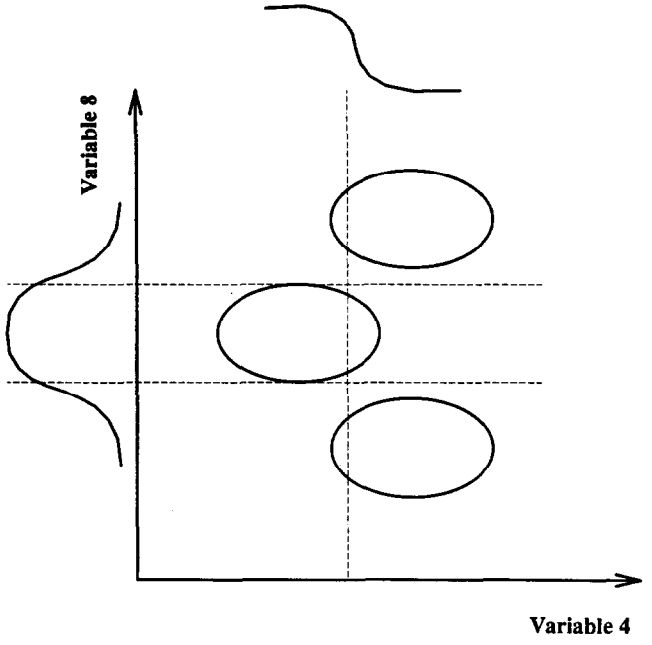

Fig. 8. Conceptual diagram of the optimal solution features for the sigmoid–sigmoid and sigmoid–Gaussian nets for problem IA.

weight of the sigmoidal function from 5.0 to 20.0, in steps of 5.0, while the offset was again fixed at 8.0. The σ parameter in the Gaussian was fixed at 2.0 for this plot.

The first system studied was the sigmoid–sigmoid net applied to data set IA. Only one feature was extracted using this network. A sigmoidal node has its response defined only with respect to a single axis. Therefore it is not necessary (or helpful) to extract more input features than there are sigmoids in the second hidden layer. In the case of the sigmoid–sigmoid net, only one feature need be extracted. Using knowledge of the sigmoid shape [17], we can definitively predict what the optimal direction for the feature extraction node in the eight-dimensional space will be. This should be parallel to axis four, and is illustrated in the conceptual diagram shown in Fig. 8. The network was trained on this problem using an epoch size of 250 randomly selected

TABLE 2

Feature vectors learned for the artificial data sets

Data set/Network *	Variable							
	1	2	3	4	5	6	7	8
IA/SS	−0.0899	−0.0373	−0.0242	−0.9819	0.0398	−0.0932	−0.1165	0.0437
IA/GS	−0.0151	−0.0485	0.0182	0.0014	0.0105	0.0076	−0.0161	0.9983
IB/SS **	−0.0925	−0.0321	−0.0321	0.9804	0.0342	−0.1075	−0.1165	0.0448
IB/GS **	−0.0114	−0.0242	−0.0188	0.0034	0.0239	0.0177	−0.0297	0.9986
IIA/GS	−0.0096	−0.0570	−0.0616	0.9897	0.0249	−0.0636	−0.0882	0.0317
IIA/GS	0.0195	−0.0149	0.0217	0.0291	0.0243	−0.0077	−0.0281	−0.9983
IIB/GS **	0.0287	0.0798	0.0570	−0.9902	−0.0500	0.0296	0.0649	−0.0378
IIB/GS **	−0.0148	0.0193	−0.0158	−0.0343	−0.0276	0.0076	0.0294	0.9982

* SS indicates a sigmoid–sigmoid net; GS indicates a Gaussian–sigmoid net.

** Back-rotated weight vector.

patterns. The stepsize and momentum learning parameters were set at 0.25 and 0.60, respectively. Two thousand training epochs were run for each feature node.

Table 2 presents the feature vectors which were 'learned' during these experiments. The columns correspond to input variables, and the rows to feature vectors. To aid comparison of the weight vectors developed in the different experiments, they are shown normalized to unit length in Table 2. The first row depicts the feature vector developed from applying the sigmoid–sigmoid net to data set IA. If the feature node was exactly parallel to axis four, then this variable would have a weight of 1.0, and all the others would have a weight of zero. While the experimental values are close to these ideal values, they are not identical. There are several reasons for this small discrepancy. First of all, the data distributions are finite in size, and not perfectly symmetrical. This may produce a small rotation of the extracted feature vector in the variable four–variable eight plane, from the ideal. Second, the values for the coordinates in the 'fully overlapped' dimensions are not identical. Therefore, some small classification power may still be present in the 'fully overlapped' dimensions. Finally, true convergence is difficult to assess, and training was in fact halted after two thousand epochs, when the error reduction rate appeared very slow.

A second network was then trained on problem IA, this time using a Gaussian classification node in the second hidden layer. This second network will be termed the 'Gaussian–sigmoid' net. Only one feature was extracted with the Gaussian–sigmoid network for problem IA. As shown in Fig. 8, only one dimension is necessary if the optimal direction in the input space is taken. Now, however, the optimal direction is parallel to axis eight, rather than axis four, as in the previous case. The normalized feature vector learned for this case is shown in the second row of Table 2. As expected, the learned feature is nearly parallel to axis eight.

To clearly illustrate the fact that Orthonet is carrying out an abstract feature extraction process, rather than a variable selection process, the variables for data set IA were mixed using a random orthonormal matrix to generate eight new variables which were orthogonal linear combinations of the original variables. The rotated data set, IB, was then analyzed for the optimal feature using the sigmoid–sigmoid and the Gaussian–sigmoid networks. For comparison purposes, the learned feature vectors were then back-transformed into the original input space, using the inverse of the mixing matrix. These back-rotated features are presented in rows 3 and 4 of Table 2. Examination of the back-rotated features reveals that equivalent results were obtained using the original and the 'mixed' (rotated) data. When comparing weight vectors in Table 2, note that a sign change carried out across a weight vector changes only the polarity of the input projections — the portion of the inputs carried on this vector remains the same.

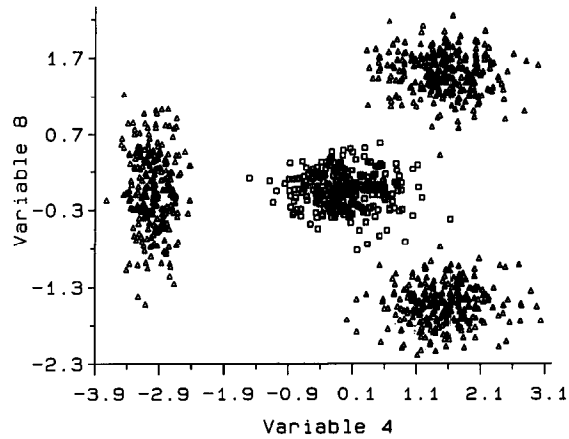

Fig. 9. Data set IIA, shown projected onto variables 4 and 8, the 'meaningful' variables for classification. The target cluster is shown with squares and interfering clusters with triangles.

Data set IIA contains an extra interfering cluster to the left of the target cluster, when projected onto axes four and eight (Fig. 9). The new interfering cluster has the same center as the target cluster along axis eight, and so would strongly overlap the target cluster, if this direction alone is considered. Therefore, the first optimal direction for the Gaussian–sigmoid classifier becomes parallel to axis four. The second optimal direction is then parallel to axis eight. The Gaussian–sigmoid network was trained on this data set to extract five features. Of course, only two mean-

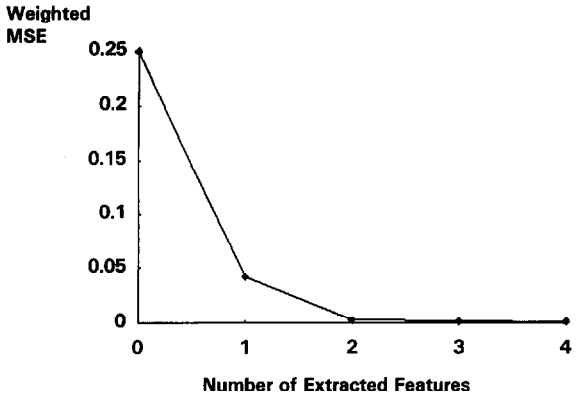

Fig. 10. The weighted mean-squared error (i.e., the network cost function), plotted as a function of the number of extracted features, when the Gaussian sigmoid net was applied to data set IIB.

ingful features exist. The coefficients for the first two extracted features are shown in rows 5 and 6 of Table 2. The predicted results were obtained. The data set IIA was then rotated to form a mixed data matrix IIB, using the same rotation matrix as before. As before, equivalent results were obtained with the rotated data set, and the back-rotated feature vectors for the first two features are presented in rows 9 and 10 of Table 2.

In a separate run, the Gaussian–sigmoid net was again trained on data set IIB, and the value of the cost function was monitored with increasing number of extracted features. Fig. 10 shows the information obtained. The leveling off of the error reduction after two features were extracted indicates, as we would expect, that the significant information is contained in the first two features.

Table 3 shows the weight matrix of the Gaussian after extracting four features from data set IIB. The last column in a Gaussian weight matrix can be considered to be an augmenting column which provides the generalization of nonzero centering for the response, i.e., translation. Orthonet was implemented to add new phantom nodes to the head of the list, so the information corresponding to the latest features is found in the upper left hand portion of the matrix. Note that the first two diagonal elements, corresponding to the last two extracted features, are very small. These values are related to the width of the response along these features, or the sensitivity to these features. Very small values there indicate a very broad, nondiscriminating response for those features. If this were a real unknown data analysis problem, this would reinforce the idea above that only the first two features carry useful information for classification with this architecture. Note also that the off-diagonal elements in the weight matrix are relatively small, indicating little rotation of the Gaussian in the feature space. This is likely due to the fact that the features are derived in stepwise fashion for the Gaussian, and hence in large part can be expected to correspond to the orthogonal axes defining the elliptical Gaussian cross-section.

In every case shown, Orthonet not only extracted (a) feature(s) which provided good separation of classes, but also one which was optimal for the specific form of the functions used in the functional mapping layers. This extension of neural network technology provides an interesting and powerful new means of performing goal-driven data reduction. The derivation of generalized backpropagation training rules for multivariate Gaussian nodes provides new flexibility in designing neural network architectures.

TABLE 3

A weight matrix for the Gaussian node in the Gaussian–sigmoid net, after extracting four features from data set IIB

	Feature 4	Feature 3	Feature 2	Feature 1	Augment
Feature 4	0.0000	0.0001	−0.0023	−0.0026	0.0000
Feature 3	−0.0001	0.0004	0.0129	−0.0123	−0.0009
Feature 2	0.0036	0.0040	1.1313	0.0433	−0.0431
Feature 1	0.0203	−0.0365	0.0682	1.4296	0.0393

CONCLUSION

The methodology described in this work represents a significant extension of numerical method capabilities available through neural networks. These methods complement, rather than supplant, other techniques. When compared to the nonlinear PLS methods, Orthonet should be applied when complex nonlinearities are suspected, or when the input data are known to contain a lot of variation that is irrelevant to the particular problem under study.

ACKNOWLEDGEMENTS

The author gratefully acknowledges the support of a National Research Council Postdoctoral Fellowship Grant while this research was carried out. In addition, the author is indebted to Stephen Stein of NIST for his frequent council, to John Travis of NIST for his assistance in deriving the generalized backpropagation rule for multivariate Gaussian nodes; and to Dave Duewer of NIST, Agnar Höskuldsson of The Danish Engineering Academy, Willem Windig of Eastman Kodak Co., and Barry Wise of Pacific Northwest Laboratories for discussions on classical multivariate techniques for orthogonal data reduction. This is an official contribution of the National Institute of Standards and Technology and thus not subject to copyright in the United States.

APPENDIX: DERIVATION OF THE LEARNING LAW FOR MULTIVARIATE GAUSSIAN NODES

The derivation is placed in the same framework used by Rumelhart and McClelland. Eqns. A1–A11 show the generalized method, and define the three differential terms which must be solved for any node. Then Eqns. A12–A29 solve for these three terms for, in addition to multivariate Gaussian nodes, linear, quadratic and sigmoidal nodes. Finally, Eqns. A30–A32 use a slight generalization of the Gaussian–sigmoid architecture from the text to illustrate the combination of the appropriate parts to obtain the learning equations.

These equations are all expressed from the viewpoint of a given network node. The reason for this is that the number of inputs, nodes, layers, and connectivity are not fixed. Furthermore, the outputs of a given node are in general the inputs to another node. Finally, the number of inputs from a node in one layer can be more or less than the number of nodes in the previous layer. The reader is cautioned therefore against trying to attach a scope for the symbols from the perspective of the overall network. The symbols are meaningful only from the perspective of an individual node. The function to be minimized is

$$E = \frac{1}{n}\frac{1}{2}\sum_{h=1}^{n}\sum_{i=1}^{m}(t_{hi} - o_{hi})^2 \tag{A1}$$

where o_{hi} is the output of the ith output node on the hth pattern; t_h is the target (correct) output of the ith output node on the hth pattern; n is the number of training patterns; m is the number of output nodes. Suppressing the pattern index yields

$$E = \tfrac{1}{2}\sum_{i=1}^{m}(t_i - o_i)^2 \tag{A2}$$

For any weight j in a node i, we wish to make 'downhill moves' proportional to the error gradient:

$$\Delta w_{ij} = \eta\frac{-\partial E}{\partial w_{ij}} \tag{A3}$$

We expand the derivative using the chain rule

$$\frac{-\partial E}{\partial w_{ij}} = \frac{-\partial E}{\partial o_i}\frac{\partial o_i}{\partial w_{ij}} = \frac{-\partial E}{\partial o_i}\frac{\partial o_i}{\partial a_i}\frac{\partial a_i}{\partial w_{ij}} \tag{A4}$$

where o_i is the output of the node i; a_i is the net input to the node i. Now proceeding further as by Rumelhart, we define a term for the ith node, δ_i as:

$$\delta_i = \frac{-\partial E}{\partial a_i} = \frac{-\partial E}{\partial o_i}\frac{\partial o_i}{\partial a_i} \tag{A5}$$

Using the δ construct, we can reexpress Eqns. 3 and 4 as

$$\frac{-\partial E}{\partial w_{ij}} = \delta_i \frac{\partial a_i}{\partial w_{ij}} \tag{A6}$$

and

$$\Delta w_{ij} = \eta \delta_i \frac{\partial a_i}{\partial w_{ij}} \tag{A7}$$

Now, if the node i is an output node, then

$$\frac{-\partial E}{\partial o_i} = (t_i - o_i) \tag{A8}$$

and

$$\delta_i = (t_i - o_i) \frac{\partial o_i}{\partial a_i} \tag{A9}$$

Else, if the node i is a hidden node, then its δ term is expressed in terms of the δ values of the nodes that it feeds:

$$\frac{-\partial E}{\partial o_i} = \sum_{k=1}^{m} \left(\frac{-\partial E}{\partial o_k} \frac{\partial o_k}{\partial o_i} \right) = \sum_{k=1}^{m} \left(\frac{-\partial E}{\partial o_k} \frac{\partial o_k}{\partial a_k} \frac{\partial a_k}{\partial o_i} \right) = \sum_{k=1}^{m} \left(\delta_k \frac{\partial a_k}{\partial o_i} \right) \tag{A10}$$

where o_i is the output of the hidden node; o_k is the output of the kth node in the layer above which takes its input from node i; a_k is the net input to the node k; m is the number of nodes taking input from the output of hidden node i; and

$$\delta_i = \left[\sum_{k=1}^{m} \left(\delta_k \frac{\partial a_k}{\partial o_i} \right) \right] \left(\frac{\partial o_i}{\partial a_i} \right) \tag{A11}$$

Until now, all expressions have been completely general. We will now begin to consider the individual forms of common node functions, so we can further substitute directly calculable expressions for the three symbolic derivatives representing the derivative of the net input to a node with respect to its weight elements and input elements, and the derivative of the node output with respect to its net input.

Net input derivatives

For any node which forms its net input by a dot product of its weight vector with its input vector, e.g., sigmoidal, linear and quadratic nodes, the equation for the net input is

$$a_i = \sum_{j=1}^{n} w_{ij} x_{ij} \tag{A12}$$

and

$$\frac{\partial a_i}{\partial w_{ij}} = x_{ij} \tag{A13}$$

and

$$\frac{\partial a_i}{\partial x_{ij}} = w_{ij} \tag{A14}$$

while for a multivariate Gaussian (MVG) node, which forms its net input as the square of the product of its weight matrix with its input vector, which we will consider to be a column vector:

$$a_i = -(\mathbf{M}_i \boldsymbol{x}_i)^{\mathrm{T}} (\mathbf{M}_i \boldsymbol{x}_i) \tag{A15}$$

where $\mathbf{M}_i$ is the weight matrix of the node i; $\boldsymbol{x}_i$ is the input vector to the node i.

By expanding the product and retaining terms sensitive to m_{ijk}, it can be shown that

$$\frac{\partial a_i}{\partial m_{ijk}} = -2 x_{ik} m_{ij.} \boldsymbol{x}_i \tag{A16}$$

where m_{ijk} is the weight in the jth row and the kth column of the weight matrix for the node i; x_{ik} is the kth element in the input vector to node i; $m_{ij.}$ is the jth row of the weight matrix for the node i; $\boldsymbol{x}_i$ is the input vector for node i; and alternatively by retaining those terms sensitive to x_{ij}

$$\frac{\partial a_i}{\partial x_{ij}} = -2(\mathbf{M}_i \boldsymbol{x}_i)^{\mathrm{T}} m_{i.j} \tag{A17}$$

where $m_{i.j}$ is the jth column of the weight matrix for the ith MVG.

Transfer function derivatives

For a linear transfer function

$$o_i = k a_i \tag{A18}$$

and

$$\frac{\partial o_i}{\partial a_i} = k \tag{A19}$$

(a slope of one can be used without loss of generality, and thereby simplifies the learning calculations), while for a simple quadratic transfer function

$$o_i = a_i^2 \tag{A20}$$

and

$$\frac{\partial o_i}{\partial a_i} = 2a_i \tag{A21}$$

For the popular sigmoidal node, determining the transfer function derivative is a bit more troublesome. For the usual sigmoidal node

$$o_i = \frac{1}{1 + e^{-a_i}} \tag{A22}$$

Substitution and the chain rule yield

$$u = 1 + e^{-a_i} \quad \Rightarrow \quad \frac{\partial o_i}{\partial a_i} = \frac{\partial o_i}{\partial u}\frac{\partial u}{\partial a_i} \tag{A23}$$

and

$$o_i = u^{-1} \quad \Rightarrow \quad \frac{\partial o_i}{\partial u} = -u^{-2} = -o_i^2 \tag{A24}$$

and

$$\frac{\partial u}{\partial a_i} = -e^{-a_i} \tag{A25}$$

It is helpful to rearrange Eqn. 22 in the following way:

$$o_i = \frac{1}{1 + e^{-a_i}} \quad \Rightarrow \quad o_i + o_i\, e^{-a_i} = 1$$

$$\Rightarrow \quad e^{-a_i} = \frac{1 - o_i}{o_i} \tag{A26}$$

Finally, substituting the results from Eqn. 26 into 25, and that result along with Eqn. 24 into Eqn. 23 yields

$$\frac{\partial o_i}{\partial a_i} = -o_i^2\left(-\frac{1 - o_i}{o_i}\right) = o_i(1 - o_i) \tag{A27}$$

Lastly, for a multivariate Gaussian node

$$o_i = e^{a_i} \tag{A28}$$

and

$$\frac{\partial o_i}{\partial a_i} = e^{a_i} = o_i \tag{A29}$$

Using these equations

It is now only necessary to select and combine the appropriate equations to express any learning rule... For an example, we will generalize slightly from the Gaussian sigmoid net used in the text, to one in which there are $n1$ external inputs, followed by $n2$ linear feature nodes, followed by $n3$ multivariate Gaussians, and $n4$ sigmoidal output nodes. All layers will be assumed to be only fully connected to the previous one.

For the sigmoidal output nodes, the learning rule is found by substituting Eqn. 27 into Eqn. 9, and that result, along with Eqn. 13, into Eqn. 7, to yield

$$\Delta w_{ij} = \eta\delta_i \frac{\partial net_i}{\partial w_{ij}} = \eta(t_i - o_i)\left[o_i(1 - o_i)\right]x_{ij} \tag{A30}$$

where w_{ij} refers to the jth weight for the ith output node; x_{ij} refers to the jth input for the ith output node. Note that except for the case of the bias input, where it will be 1.0, this will be the output of the jth Gaussian; while for the Gaussian nodes used in the second hidden layer, substitute Eqn. 29 and 14 into Eqn. 11, and this result along with Eqn. 16 into Eqn. 7 to yield

$$\Delta m_{ijk} = -2\eta\left(\sum_{l=1}^{n4}(\delta_l w_{il})\right)o_i x_{ik} m_{ij.}x_i \tag{A31}$$

where m_{ijk} refers to the kth weight in the jth row of the ith Gaussian; δ_l refers to the δ value for the lth output node; w_{il} refers to the weight between the ith MVG and the lth output node; $m_{ij.}$ is the jth row of the weight matrix for the MVG node i.

Finally, for the feature nodes used in the first hidden layer,

$$\Delta w_{ij} = -2\sum_{k=1}^{n3}\left[\delta_k(\mathbf{M}_k o_i)^{\mathrm{T}} m_{k.i}\right] \tag{A32}$$

where w_{ij} refers to the jth weight for the ith feature node; δ_k refers to the δ value for the kth multivariate Gaussian; $\mathbf{M}_k$ refers to the weight matrix for the kth MVG; o_i refers to the output of the ith feature node; $m_{k.i}$ refers to the ith column of the weight matrix for the kth MVG node.

As noted above and in the text, the derivatives are actually averaged for *batchlen* input patterns, where *batchlen* is between 1 and the number of input patterns, before changing the weights.

While these equations might appear complicated to implement, the beauty of neural networks using the δ construct is the simplicity of computer encoding of the algorithm. All of the calculations are private to a given node. Therefore, a given node does not need to 'know' how its values on the input lines were calculated on the forward pass, or how the delta values received on the output lines were calculated on the backward pass. The computer code for any node only needs to have the definitions of the three derivatives associated with it. All that remains is some very simple code to allow passing values along the connections.

REFERENCES

1 P.R. Griffiths, *Chemical Infrared Fourier Transform Spectroscopy*, Wiley, New York, 1975.

2 E.R. Malinowski, *Factor Analysis in Chemistry*, Wiley, New York, 1991.

3 A. Höskuldsson, PLS regression methods, *Journal of Chemometrics*, 2 (1988) 211–228.

4 A. Höskuldsson, The H-principle in modelling with applications to chemometrics, *Chemometrics and Intelligent Laboratory Systems*, 14 (1992) 139–153.

5 S. Wold, N. Kettaneh-Wold and B. Skagerberg, Nonlinear PLS modeling, *Chemometrics and Intelligent Laboratory Systems*, 7 (1989) 53–65.

6 S. Wold, Nonlinear partial least squares modelling II. Spline inner relation, *Chemometrics and Intelligent Laboratory Systems*, 14 (1992) 71–84.

7 O.M. Kvalheim, Latent variable regression models with higher-order terms: an extension of response modeling by orthogonal design and multiple linear regression, *Chemometrics and Intelligent Laboratory Systems*, 8 (1990) 59–67.

8 I.E. Frank, A nonlinear PLS model, *Chemometrics and Intelligent Laboratory Systems*, 8 (1990) 109–119.

9 V.M. Taavitsainen and P. Korhonen, Nonlinear data analysis with latent variables, *Chemometrics and Intelligent Laboratory Systems*, 14 (1992) 185–194.

10 A. Höskuldsson, Quadratic PLS regression, *Journal of Chemometrics*, 6 (1992) 307–334.

11 T. Hrycej, Backward inhibition: a mechanism for unsupervised discovery of input features, *Journal of Cognitive Systems*, 2, 3 (1989) 233–250.

12 T. Hrycej, Supporting supervised learning by self-organization, *Neurocomputing*, 4 (1992) 17–30.

13 S.J. Qin and T.J. McAvoy, Nonlinear PLS modeling using neural networks, *Computers and Chemical Engineering*, 16 (1992) 379–391.

14 T.R. Holcomb and M. Morari, Nonlinear PLS modeling using neural networks, *Computers and Chemical Engineering*, 16 (1992) 393–411.

15 L.J. Bellamy, *The Infrared Spectra of Complex Molecules*, Chapman and Hall, London, 1975.

16 B.M. Wise and N.L. Ricker, Identification of finite impulse response models with continuum regression, *Journal of Chemometrics*, 7 (1993) 1–14.

17 B.J. Wythoff, Backpropagation neural networks: a tutorial, *Chemometrics and Intelligent Laboratory Systems*, 18 (1993) 115–155.

18 J. Zupan and J. Gasteiger, Neural networks: a new method for solving chemical problems or just a passing phase?, *Analytica Chimica Acta*, 248 (1991) 1–30.

19 D.E. Rumelhart and J.L. McClelland, *Parallel Distributed Processing*, Vol. 1, MIT Press, Cambridge, MA, 1986.

20 J. Hertz, A. Krogh and R.G. Palmer, *Introduction to the Theory of Neural Computation*, Addison-Wesley, New York, 1991.

21 P. Gemperline, J.R. Long and V.G. Gregoriou, Nonlinear multivariate calibration using principal components regression and artificial neural networks, *Analytical Chemistry*, 63 (1991) 2313–2323.

22 D. Coomans, D.L. Massart, I. Broeckart and A. Tassin, Potential methods in pattern recognition: Part 1. Classification aspects of the supervised method ALLOC, *Analytica Chimica Acta*, 133 (1981) 215–224.

23 J. Moody and C. Darken, Learning with localized receptive fields, in D. Touretzky, G. Hinton and T. Sejnowski (Editors), *Proceedings of the 1988 Connectionist Models Summer School*, Morgan Kaufmann, San Mateo, CA, 1989, pp. 133–143.

24 A. Saha, J. Christian, D.S. Tang and C.L. Wu, Oriented non-radial basis functions for image coding and analysis, in R.P. Lippman, J.E. Moody and D.S. Touretzky (Editors), *Advances in Neural Information Processing Systems*, Vol. 3, Morgan Kaufmann, San Mateo, CA, 1991.

25 R.O. Hill, Jr., *Elementary Linear Algebra*, Academic Press, New York, 1986.

26 B. Curry and D.E. Rumelhart, MSnet: a neural network which classifies mass spectra, *Tetrahedron Computer Methodology*, (1990) 213–237.